

many_controller 第三方数值库：依赖关系、源码组织、安装与 CMake 查找全解

对应当前工程的 3rdpart/、px4ctrl/CMakeLists.txt 与构建脚本

2026 年 8 月 11 日

目录

1 文档目的与结论先行	2
2 首先区分：源码工程、库、插件、工具和独立程序	2
2.1 库并不等于独立软件	2
2.2 当前安装脚本产生什么	3
3 第三方源码进入工程的六种方式	3
3.1 方式一：操作系统开发包	3
3.2 方式二：主仓库顶层 Git 子模块	4
3.3 方式三：上游项目自己的嵌套 Git 子模块	4
3.4 方式四：上游仓库直接包含第三方源码	4
3.5 方式五：配置阶段下载并构建	4
3.6 方式六：项目自己的生成源码	5
4 Coin-OR、Ipopt 与 CasADi 的准确关系	5
4.1 Coin-OR 是组织和生态，不是单个库	5
4.2 Ipopt 自己包含什么	5
4.3 CasADi 是符号框架加插件体系	6
5 acados 的内部关系	6
6 当前启用的完整原生数值依赖图	7
6.1 从系统底层到最终节点	7
6.2 关键系统稀疏库的实际传递关系	9
7 构建顺序：依赖图不是唯一线性队列	9

7.1	第 0 层：构建工具和系统库	9
7.2	第 1 层：叶子源码库	9
7.3	第 2 层：中间求解器	9
7.4	第 3 层：上层框架	10
7.5	第 4-5 层：生成代码和 ROS 2 目标	10
8	固定版本、源码和产物总表	10
9	安装前缀：所谓“根目录”究竟是什么	11
9.1	不要把安装前缀和文件系统根目录混淆	11
9.2	安装端变量：CMAKE_INSTALL_PREFIX	11
9.3	查找端变量：CMAKE_PREFIX_PATH	12
10	标准 CMake 包的推荐查找方式	12
10.1	优先使用 find_package 和导入目标	12
10.2	包级变量 Foo_DIR	13
10.3	不建议在项目内硬编码个人路径	13
11	没有标准包配置时的 CMake 处理	13
11.1	find_path 与 find_library	13
11.2	把手工结果封装为导入目标	14
11.3	CMAKE_INCLUDE_PATH 与 CMAKE_LIBRARY_PATH	14
12	直接把源码纳入当前构建	15
12.1	标准 CMake 子项目：add_subdirectory	15
12.2	明确列出普通源文件：add_library	15
12.3	构建时需要独立工具：find_program 与 custom command	16
13	特殊项目必须按各自风格处理	16
13.1	Ipopt: Autotools 加系统 MUMPS	16
13.2	OSQP: 固定本地 QDLDL	16
13.3	acados: 嵌套源码和双重安装变量	17
13.4	CasADi: 插件选项决定真实依赖	17
13.5	Eigen: 只有头文件，不应寻找 libEigen.so	17
13.6	NLopt: 上游库名与发行版拆包可能不同	18
13.7	SuiteSparse: 使用发行版传递依赖	18

14 自定义前缀的完整示例	18
14.1 统一安装到单独目录	18
14.2 运行时共享库搜索	19
14.3 工具链文件统一管理前缀	19
15 当前 CMake 目标与后端耦合	19
15.1 三个最终节点	19
15.2 为什么只运行 acados 仍要安装 Ipopt/NLopt/CasADi	20
16 Python 计算、代码生成和仿真依赖	20
16.1 MuJoCo 仿真节点	20
16.2 噪声分析脚本	20
16.3 acados 重新生成环境	21
17 存在于源码但当前没有启用的计算库	21
17.1 CasADi 可选生态	21
17.2 acados 可选后端	21
17.3 Eigen 测试中出现的库	21
18 常见错误的根因与处理	21
18.1 导入目标引用不存在的绝对路径	21
18.2 QDLDL 更新步骤 rebase 失败	22
18.3 Eigen/Eigen 找不到	22
18.4 qpOASES.hpp 找不到	22
18.5 GLIBC 符号版本不匹配	22
18.6 编译能找到，运行时找不到	23
19 新机器从零构建流程	23
19.1 安装系统依赖	23
19.2 同步固定源码	23
19.3 安装到 /usr/local	23
19.4 编译工作空间	23
20 验证依赖是否正确	24
20.1 确认子模块提交	24
20.2 确认 CMake 包配置	24
20.3 确认头文件和共享库	24

20.4 确认最终程序没有引用仓库旧二进制	24
21 推荐的长期工程结构	24
21.1 保持源码、构建结果和安装结果分离	24
21.2 每个依赖明确记录四个信息	25
21.3 将后端按需编译	25
22 变量速查表	25
23 最终原则	26

1 文档目的与结论先行

本文统一回答以下问题：

1. 工程中哪些对象是库，哪些是独立程序或构建工具；
2. 静态库、共享库、纯头文件库、插件和普通源文件如何进入最终程序；
3. Coin-OR、Ipopt、CasADi、acados、OSQP 等项目之间到底是什么关系；
4. 哪些依赖来自顶层 Git 子模块，哪些来自嵌套子模块，哪些源码直接包含在上游仓库中，哪些来自操作系统；
5. 所有当前启用的原生数值库、Python 计算库及其完整依赖顺序；
6. 安装到 `/usr/local` 与安装到自定义目录时，CMake 应设置哪些变量；
7. 对不遵守标准 CMake 安装结构的库，应如何按其自身构建风格处理；
8. 如何在另一台计算机同步源码、重新编译、排查找不到头文件或共享库的问题。

最重要的结论是：这些库并不是彼此双向耦合，而是组成一张**有向无环依赖图**。例如 OSQP 依赖 QDLDL，但 QDLDL 不依赖 OSQP；acados 依赖 HPIPM 和 BLASFEO，但它不依赖 Ipopt；CasADi 的 Ipopt 插件依赖 Ipopt，但 Ipopt 不依赖 CasADi。

当前工程看起来“所有库都必须存在”，主要是因为 `px4ctrl_node` 同时编译了多个控制器后端，而不是因为这些求解器天然相互依赖。

2 首先区分：源码工程、库、插件、工具和独立程序

2.1 库并不等于独立软件

一个上游 Git 仓库可能同时产生库、命令程序、测试程序、插件和开发配置。因此，“这个项目是软件还是库”经常不是二选一。

类型	常见产物	如何被工程使用	是否进入最终可执行文件
纯头文件库	<code>Eigen/*.h</code>	编译器实例化模板	使用到的机器码直接进入目标
静态库	<code>libfoo.a</code>	链接阶段抽取目标文件	被使用的代码通常进入目标
共享库	<code>libfoo.so</code>	链接时记录依赖，运行时加载	代码通常不复制进目标

类型	常见产物	如何被工程使用	是否进入最终可执行文件
插件库	libcasadi_nlpso1_ipopt	库按名称动态加载	作为独立共享对象运行
普通源码	model.c、solver.cpp	add_library/add_executable	直接编译进入对应目标
构建工具	CMake、编译器、t_renderer	构建时生成或转换文件	不进入运行时程序
独立程序	demo、CLI、ROS 2 node	由用户或系统直接执行	本身就是最终可执行文件
包配置	FooConfig.cmake、foo.pc	告诉构建系统头文件和库的位置	不含算法代码

因此，“独立软件不可能被编译进另一个程序”只对**已经形成的可执行文件**成立。库、插件、普通源码和模板头文件都可以以不同方式参与另一个可执行文件的构建。

2.2 当前安装脚本产生什么

3rdpart/build_all.sh 主要安装开发库，而不是面向用户的独立应用：

- Eigen：头文件和 CMake/package 配置；
- OSQP、NLOPT、Ipopt、CasADi、acados、HPIPM、BLASFEO：共享库、头文件和包配置；
- QDLDL：作为 OSQP 的内部构建输入，不单独安装；
- OSQP demo、NLOPT Python/Matlab/Octave 绑定、Ipopt Java、acados examples：当前均关闭；
- px4ctrl 专用 acados solver：由主工程从 C 源码编译成静态目标，不作为通用第三方库安装。

3 第三方源码进入工程的六种方式

3.1 方式一：操作系统开发包

发行版把头文件、共享库、静态库和元数据安装到标准位置，例如：

```
/usr/include/
/usr/include/suitesparse/
/usr/lib/x86_64-linux-gnu/
/usr/lib/x86_64-linux-gnu/pkgconfig/
```

当前系统来源的主要数值库为 BLAS、LAPACK、MUMPS sequential、Scotch、METIS 和 SuiteSparse/UMFPACK。它们由 apt 管理，不复制到仓库。

3.2 方式二：主仓库顶层 Git 子模块

主仓库只记录上游仓库 URL 和精确提交：

```
3rdpart/src/eigen
3rdpart/src/qdldl
3rdpart/src/osqp
3rdpart/src/nlopt
3rdpart/src/ipopt
3rdpart/src/casadi
3rdpart/src/acados
```

这些目录在 Git 中是 mode 160000 的 gitlink，而不是把几万个上游文件复制进主仓库历史。另一台机器必须执行 `git submodule update --init` 才会得到实际源码。

3.3 方式三：上游项目自己的嵌套 Git 子模块

acados 在自己的仓库中记录 HPIPM、BLASFEO 等外部项目。当前只初始化实际使用的两个：

```
git -C 3rdpart/src/acados submodule update --init \
    external/blasfeo external/hpipm
```

这表示主仓库固定 acados 提交，acados 提交再固定 HPIPM/BLASFEO 提交，形成两级版本锁定。

3.4 方式四：上游仓库直接包含第三方源码

CasADi 当前没有实际 Git 子模块，但其源码树中直接包含若干第三方源码快照，例如：

```
casadi/external_packages/casadi-sundials
casadi/external_packages/CSparse
casadi/external_packages/tinyxml2-9.0.0
casadi/external_packages/qpoases
```

这种方式对 Git 来说只是 CasADi 仓库的普通文件。是否编译仍由 CasADi 的 CMake 选项决定；“源码存在”不等于“当前启用”。

3.5 方式五：配置阶段下载并构建

CasADi 可以通过 `WITH_BUILD_*` 选项让 CMake 的 `ExternalProject` 在构建目录中下载依赖；OSQP 也会通过 `FetchContent` 获取 QDLDL。优点是自动，缺点是：

- 构建时依赖网络；

- 下载源码通常留在临时 build 目录；
- 更新步骤可能把标签误当成远端分支；
- 版本、补丁和失败位置不够直观。

当前工程尽量避免这种隐式下载。OSQP 通过以下变量强制使用已经锁定的 QDLDL 源码：

```
-DFETCHCONTENT_SOURCE_DIR_QDLDL=/path/to/3rdpart/src/qdlldl
-DFETCHCONTENT_UPDATES_DISCONNECTED=ON
```

3.6 方式六：项目自己的生成源码

acados Python 模板根据本项目动力学、代价和约束生成 C/H/JSON。这些文件属于 px4ctrl，而不是通用 acados 安装包：

```
src/realflight_modules/px4ctrl/generated/acados/px4ctrl_nmpc
```

生成源码提交到 Git；.o、.a、.so 不提交。CMake 在每台机器上重新编译这些 C 文件。

4 Coin-OR、Ipopt 与 CasADi 的准确关系

4.1 Coin-OR 是组织和生态，不是单个库

Coin-OR (Computational Infrastructure for Operations Research) 包含许多独立项目：

```
Coin-OR ecosystem
|- Ipopt      nonlinear programming
|- Clp       linear programming
|- Cbc       mixed-integer programming
|- Bonmin    mixed-integer nonlinear programming
|- Cgl       cut generation
|- CoinUtils shared utilities
`- ThirdParty-* build adapters for external packages
```

它们分别有独立仓库和库文件，例如 libipopt.so、libCbc.so、libClp.so。目录 include/coin-or 只是头文件安装约定，不表示存在名为“coin-or”的单体库。

4.2 Ipopt 自己包含什么

当前固定的 Ipopt 仓库没有 Git 子模块。它包含：

- 内点法核心算法；
- TNLP、C/C++ 接口；
- 对 MUMPS、HSL、Pardiso、SPRAL 等线性求解器的适配代码；

- Autotools 构建系统。

但是“包含某求解器的适配接口”不等于“包含该求解器的实现”。当前 MUMPS、BLAS 和 LAPACK 实现来自系统包：

```
Ipopt source
-> Ipopt MUMPS adapter
-> system libdmumps_seq.so
-> system BLAS/LAPACK/Scotch/libgfortran
```

HSL、Pardiso 和 SPRAL 等后端当前没有启用。

4.3 CasADi 是符号框架加插件体系

CasADi 提供符号表达式、自动微分、函数图、代码生成和求解器统一接口。它不是 Ipopt 或 OSQP 本身，而是通过插件调用它们：

```
CasADi core
|- nlpsol_ipopt plugin -> Ipopt
|- conic_osqp plugin   -> OSQP      (currently disabled)
|- conic_qpoases       -> qpOASES   (currently disabled)
|- linsol_csparse      -> CSparse
`- integrator_sundials -> Sundials
```

当前构建明确设置：

```
-DWITH_IPOPT=ON
-DWITH_BUILD_IPOPT=OFF
-DWITH_SELFCONTAINED=OFF
-DWITH_PYTHON=OFF
```

含义是启用 CasADi 的 Ipopt 插件，但使用此前安装到目标前缀的 Ipopt，不让 CasADi 重新下载另一份 Ipopt。

CasADi 3.7.2 默认还编译仓库内置 Sundials、CSparse 和 TinyXML。当前 px4ctrl 代码实际只调用：

```
casadi::nlpsol("px4ctrl_casadi_nmpc", "ipopt", nlp, options)
```

因此 Sundials、CSparse、TinyXML 是由 CasADi 默认配置带入的组件，不是当前控制器算法的直接需要。后续可在完整测试后考虑关闭：

```
-DWITH_SUNDIALS=OFF
-DWITH_CSPARSE=OFF
-DWITH_TINYXML=OFF
```

5 acados 的内部关系

acados 是独立的实时最优控制框架。当前配置使用的求解链为：

```
generated px4ctrl OCP solver
-> acados SQP / OCP-NLP interface
-> HPIPM partial-condensing QP solver
-> BLASFEO dense linear algebra kernels
-> libc / libm
```

acados、HPIPM、BLASFEO 的源码关系和安装产物为：

项目	源码位置	安装产物	当前启用
acados	顶层 Git 子模块	libacados.so	是
HPIPM	acados 内部子模块	libhpiPM.so	是
BLASFEO	acados 内部子模块	libblasfeo.so	是
qpOASES/DAQP/OSQP等	acados 可选内部项目	对应插件或库	否

当前 ELF 运行时依赖关系为：

```
libacados.so
|- libhpiPM.so
|- libblasfeo.so
|- libm.so
`- libc.so

libhpiPM.so
|- libblasfeo.so
`- libc.so
```

6 当前启用的完整原生数值依赖图

6.1 从系统底层到最终节点

下面只列当前真正参与构建或运行的数值/计算库，不把上游源码中关闭的测试和插件误算为必需项：

```
System foundation
|- libc / libm / libstdc++ / pthread
|- BLAS
|- LAPACK -> BLAS
|- Scotch
|- METIS
|- libgfortran
`- SuiteSparseConfig
```

System sparse solvers

```
| - MUMPS sequential
|   | - BLAS
|   | - LAPACK
|   | - Scotch
|   ` - libgfortran
|
` - SuiteSparse
    | - AMD -> SuiteSparseConfig
    | - COLAMD / CAMD / CCOLAMD
    | - CHOLMOD
    |   | - AMD/COLAMD/CAMD/CCOLAMD
    |   | - METIS
    |   | - BLAS/LAPACK
    |   ` - OpenMP runtime
    ` - UMFPACK
        | - AMD
        | - CHOLMOD
        | - SuiteSparseConfig
        ` - BLAS
```

Project source leaves

```
| - Eigen (header-only)
| - QDLDL
| - NLOpt
` - BLASFEO
```

Intermediate solvers

```
| - OSQP -> QDLDL + libm
| - HPIPM -> BLASFEO
` - Ipopt -> MUMPS -> BLAS/LAPACK/Scotch
```

Upper frameworks

```
| - acados -> HPIPM -> BLASFEO
` - CasADi -> Ipopt plugin -> Ipopt -> MUMPS
```

Project generated code

```
` - px4ctrl_acados_ocp_solver -> acados + HPIPM + BLASFEO
```

Final executables

```
| - px4ctrl_node
|   | - Eigen
|   | - OSQP -> QDLDL
|   | - NLOpt
|   | - Ipopt -> MUMPS -> BLAS/LAPACK/Scotch
```

```
|  |- CasADi -> Ipopt
|  `-- acados solver -> acados -> HPIPM -> BLASFEO
|- gpttraj_visualizer_node -> Eigen + OSQP
`-- px4ctrlrate_node -> Eigen + UMFPACK -> SuiteSparse
```

6.2 关键系统稀疏库的实际传递关系

在当前 Ubuntu 环境中, `libumfpack.so` 的动态依赖包含 `libamd`、`libcholmod`、`libsuitesparseconfig` 和 `BLAS`; `libcholmod.so` 又继续依赖 `COLAMD`、`CAMD`、`CCOLAMD`、`METIS`、`LAPACK`、`BLAS` 和 `OpenMP runtime`。因此只在主工程写 `-lumfpack`, 不代表它在运行时只有一个库。

正确做法是安装发行版的 `libsuitesparse-dev`, 让包管理器维护这条传递链, 而不是手工复制单个 `libumfpack.so`。

7 构建顺序：依赖图不是唯一线性队列

真正的约束是“依赖必须先于使用者完成”, 独立分支可以并行。一个合法的拓扑顺序如下。

7.1 第 0 层：构建工具和系统库

```
gcc/g++/gfortran
CMake + Make
Autoconf + Automake + Libtool
pkg-config
BLAS + LAPACK
MUMPS sequential + Scotch
METIS
SuiteSparse/UMFPACK
```

7.2 第 1 层：叶子源码库

以下项目互相独立, 可并行准备:

```
Eigen
QDLDL
NLOpt
BLASFEO
```

7.3 第 2 层：中间求解器

```
OSQP <- QDLDL
HPIPM <- BLASFEO
Ipopt <- system MUMPS + BLAS/LAPACK
```

7.4 第 3 层：上层框架

```
acados <- HPIPM + BLASFEO
CasADi <- installed Ipopt
```

acados 和 CasADi 互不依赖，可以并行。当前脚本串行执行只是为了输出清晰和失败即停。

7.5 第 4–5 层：生成代码和 ROS 2 目标

```
px4ctrl generated acados C sources
-> px4ctrl_acados_ocp_solver static target
-> px4ctrl_node

other branches
-> gpttraj_visualizer_node
-> px4ctrlrate_node
```

当前 build_all.sh 的线性执行顺序为：

```
Eigen
-> OSQP (uses local QDLDL)
-> NLopt
-> Ipopt
-> CasADi
-> acados (internally BLASFEO, then HPIPM)
-> colcon build px4ctrl
```

8 固定版本、源码和产物总表

项目	固定版本	源码来源	主要安装产物	类型
Eigen	3.4.0	顶层子模块	include/eigen3、CMake config	纯头文件
QDLDL	0.1.8	顶层子模块	编译进 OSQP	C 源码/对象
OSQP	1.0.0	顶层子模块	libosqp.so	共享库
NLopt	2.7.1	顶层子模块	libnlopt.so	共享库
Ipopt	3.14.11	顶层子模块	libipopt.so	共享库
CasADi	3.7.2	顶层子模块	libcasadi.so、插件	核心 + 插件库
acados	4c23274e4	顶层子模块	libacados.so	共享库

项目	固定版本	源码来源	主要安装产物	类型
HPIPM	acados 锁定	acados 内部子模块	libhpipm.so	共享库
BLASFEO	acados 锁定	acados 内部子模块	libblasfeo.so	共享库
MUMPS	发行版版本	系统包	libdmumps_seq.so 等	共享库
SuiteSparse	发行版版本	系统包	UMFPACK/CHOLMOD/ARPACK 等	库集合
px4ctrl solver	工程生成版本	主工程普通源码	静态 CMake target	项目专用

9 安装前缀：所谓“根目录”究竟是什么

9.1 不要把安装前缀和文件系统根目录混淆

Linux 文件系统根目录是 /。普通源码安装不建议直接设置：

```
-DCMAKE_INSTALL_PREFIX=/
```

因为这样会把文件直接散落到 /include、/lib、/bin，难以维护。通常所说“像普通安装一样装到系统里”，指的是安装到系统本地软件前缀：

```
/usr/local
|- include/
|- lib/
|- lib64/
|- bin/
|- share/
`- cmake/ or lib/cmake/
```

/usr/local 本质上也是普通目录，只是编译器、CMake 和动态加载器通常默认认识它。

9.2 安装端变量：CMAKE_INSTALL_PREFIX

对标准 CMake 项目，安装位置在编译第三方库时设置：

```
cmake -S source -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX=/usr/local
cmake --build build --parallel 2
sudo cmake --install build
```

安装到自定义目录：

```
cmake -S source -B build \
  -DCMAKE_INSTALL_PREFIX=/opt/manycontroller-deps
cmake --build build --parallel 2
```

```
sudo cmake --install build
```

注意：CMAKE_INSTALL_PREFIX 回答的是“把当前正在编译的库安装到哪里”，不是“去哪里查找依赖”。

9.3 查找端变量：CMAKE_PREFIX_PATH

主工程需要查找已经安装的库时使用：

```
colcon build --packages-up-to px4ctrl --cmake-clean-cache \  
--cmake-args \  
-DCMAKE_PREFIX_PATH=/opt/manycontroller-deps
```

多个前缀用 CMake 列表分隔：

```
-DCMAKE_PREFIX_PATH="/opt/deps-a;/opt/deps-b"
```

两者关系可记为：

CMAKE_INSTALL_PREFIX：安装到哪里，	CMAKE_PREFIX_PATH：去哪里寻找
-----------------------------	-------------------------

 (1)

10 标准 CMake 包的推荐查找方式

10.1 优先使用 find_package 和导入目标

标准包安装后会提供 FooConfig.cmake 和导入目标：

```
find_package(Eigen3 3.3 CONFIG REQUIRED)  
find_package(osqp 1.0 CONFIG REQUIRED)  
find_package(Nlopt 2.7 CONFIG REQUIRED)  
find_package(casadi CONFIG REQUIRED)  
find_package(acados CONFIG REQUIRED)  
  
target_link_libraries(my_target PRIVATE  
  Eigen3::Eigen  
  osqp::osqp  
  Nlopt::nlopt  
  casadi::casadi  
  acados  
)
```

导入目标不仅表示一个 .so，还携带：

- 头文件目录；
- 编译宏和编译选项；
- 传递链接库；

- Debug/Release 对应库；
- 必要时的系统依赖。

因此应优先链接 `osqp::osqp`，而不是手工写 `-losqp`。

10.2 包级变量 Foo_DIR

如果某个包的配置文件位置特殊，可以精确指定：

```
-DEigen3_DIR=/opt/deps/share/eigen3/cmake
-Dosqp_DIR=/opt/deps/lib/cmake/osqp
-DNlopt_DIR=/opt/deps/lib/cmake/nlopt
-Dcasadi_DIR=/opt/deps/lib/cmake/casadi
-Dacados_DIR=/opt/deps/cmake
```

`Foo_DIR` 应指向**包含 `FooConfig.cmake` 的目录**，不是安装前缀，也不是共享库文件本身。

包配置名称大小写由上游决定，例如当前 `Nlopt` 使用 `NloptConfig.cmake` 和 `Nlopt::nlopt`。

10.3 不建议在项目内硬编码个人路径

不推荐：

```
set(casadi_DIR "/home/manifold/project/3rdpart/casadi/cmake")
```

推荐由用户或工具链文件在配置时传入：

```
cmake ... -DCMAKE_PREFIX_PATH=/opt/manycontroller-deps
```

这样同一份 `CMakeLists` 可以在不同用户名、架构和安装目录下复用。

11 没有标准包配置时的 CMake 处理

11.1 find_path 与 find_library

`Ipopt` 和 `UMFPACK` 当前采用显式查找：

```
find_path(IPOPT_INCLUDE_DIR
  NAMES IpIpoptApplication.hpp
  PATH_SUFFIXES coin-or coin
  REQUIRED
)
find_library(IPOPT_LIBRARY NAMES ipopt REQUIRED)

find_path(UMFPACK_INCLUDE_DIR
  NAMES suitesparse/umfpack.h
  REQUIRED)
```



```
)  
find_library(UMFPACK_LIBRARY NAMES umfpack REQUIRED)
```

自定义前缀可通过 CMAKE_PREFIX_PATH 影响这些命令，也可以提供项目自己的 cache 变量：

```
set(IPOPT_ROOT "" CACHE PATH "Ipopt installation prefix")  
  
find_path(IPOPT_INCLUDE_DIR  
  NAMES IpIpoptApplication.hpp  
  HINTS "${IPOPT_ROOT}"  
  PATH_SUFFIXES include/coin-or include/coin  
  REQUIRED  
)  
  
find_library(IPOPT_LIBRARY  
  NAMES ipopt  
  HINTS "${IPOPT_ROOT}"  
  PATH_SUFFIXES lib lib64  
  REQUIRED  
)
```

用户调用：

```
cmake ... -DIPOPT_ROOT=/opt/ipopt
```

11.2 把手工结果封装为导入目标

为了保留现代 CMake 的传递关系，可以进一步封装：

```
add_library(Ipopt::Ipopt UNKNOWN IMPORTED)  
set_target_properties(Ipopt::Ipopt PROPERTIES  
  IMPORTED_LOCATION "${IPOPT_LIBRARY}"  
  INTERFACE_INCLUDE_DIRECTORIES "${IPOPT_INCLUDE_DIR}"  
)  
  
target_link_libraries(my_target PRIVATE Ipopt::Ipopt)
```

实际项目若需要将 MUMPS 等传递库明确写入接口，也应加到 INTERFACE_LINK_LIBRARIES，而不是在所有可执行文件中重复。

11.3 CMAKE_INCLUDE_PATH 与 CMAKE_LIBRARY_PATH

它们是比 CMAKE_PREFIX_PATH 更低层的全局提示：

```
-DCMAKE_INCLUDE_PATH=/opt/deps/include  
-DCMAKE_LIBRARY_PATH=/opt/deps/lib
```

适合没有规范安装结构的遗留库，但无法告诉 CMake 哪些头文件和库属于同一版本，也不能表达传递依赖。因此优先级应为：

1. CMAKE_PREFIX_PATH;
2. 包级 Foo_DIR;
3. 项目定义的 Foo_ROOT;
4. 最后才是 CMAKE_INCLUDE_PATH/CMAKE_LIBRARY_PATH。

12 直接把源码纳入当前构建

12.1 标准 CMake 子项目：add_subdirectory

如果第三方项目支持作为子目录使用：

```
set(BUILD_TESTING OFF CACHE BOOL "" FORCE)
add_subdirectory(
  "${THIRDPARTY_SOURCE_DIR}/foo"
  "${CMAKE_BINARY_DIR}/third_party/foo"
)
target_link_libraries(my_target PRIVATE foo::foo)
```

风险包括 cache 选项冲突、目标重名、全局编译选项污染和安装规则混入主工程。因此大型项目如 Ipopt、CasADi、acados 更适合独立安装。

12.2 明确列出普通源文件：add_library

项目生成代码适合直接编译：

```
set(GENERATED_DIR
  "${CMAKE_CURRENT_SOURCE_DIR}/generated/acados/px4ctrl_nmpc"
)

add_library(px4ctrl_acados_ocp_solver STATIC
  "${GENERATED_DIR}/acados_solver_px4ctrl_nmpc.c"
  "${GENERATED_DIR}/px4ctrl_nmpc_model/model.c"
  "${GENERATED_DIR}/px4ctrl_nmpc_cost/cost.c"
)

target_include_directories(px4ctrl_acados_ocp_solver PUBLIC
  "${GENERATED_DIR}"
)

target_link_libraries(px4ctrl_acados_ocp_solver PUBLIC
  acados hpipm blasfeo m
```

```
)
```

这里 `find_package` 负责找到已经安装的 `acados`; `add_library` 负责真正编译项目生成的 C 文件。两者职责不同。

12.3 构建时需要独立工具: `find_program` 与 `custom command`

如果需要代码生成器, 不应把可执行文件传给 `target_link_libraries`, 而应:

```
find_program(T_RENDERERER_EXECUTABLE NAMES t_renderer REQUIRED)

add_custom_command(
  OUTPUT generated_solver.c
  COMMAND "${T_RENDERERER_EXECUTABLE}" template.in generated_solver.c
  DEPENDS template.in
  VERBATIM
)
```

这清楚地区分了“构建时运行的工具”和“运行时链接的库”。

13 特殊项目必须按各自风格处理

13.1 Ipopt: Autotools 加系统 MUMPS

Ipopt 不是标准 CMake 子项目。当前流程是:

```
mkdir -p build/ipopt
cd build/ipopt
/path/to/ipopt/configure \
  --prefix=/usr/local \
  --disable-java \
  "--with-mumps-cflags=-I/usr/include -I/usr/include/mumps_seq" \
  --with-mumps-lflags=-ldmumps_seq \
  "--with-lapack-lflags=-llapack -lblas"
make -j2
sudo make install
```

Ubuntu 的 sequential MUMPS 头文件同时分布在 `/usr/include` 和 `/usr/include/mumps_seq`。遗漏后者会导致 MPI stub 头文件找不到; 没有安装 `libmumps-seq-dev` 会出现 `mumps_compat.h`、`dmumps_c.h` 等缺失。

13.2 OSQP: 固定本地 QDLDL

OSQP 1.0 的 builtin algebra backend 使用 QDLDL。当前不让 `FetchContent` 自己更新远端:

```
cmake -S 3rdpart/src/osqp -B 3rdpart/build/osqp \
```

```
-DCMAKE_INSTALL_PREFIX=/usr/local \
-DOSQP_BUILD_SHARED_LIB=ON \
-DOSQP_BUILD_STATIC_LIB=OFF \
-DFETCHCONTENT_UPDATES_DISCONNECTED=ON \
-DFETCHCONTENT_SOURCE_DIR_QDLDL="$PWD/3rdpart/src/qdldl"
```

这样不会再发生更新脚本尝试对不存在的 origin/v0.1.8 分支 rebase。

13.3 acados: 嵌套源码和双重安装变量

acados 构建前必须初始化 HPIPM/BLASFEO。当前配置:

```
cmake -S 3rdpart/src/acados -B 3rdpart/build/acados \
-DMAKE_INSTALL_PREFIX=/usr/local \
-DACADOS_INSTALL_DIR=/usr/local \
-DBUILD_SHARED_LIBS=ON \
-DBLASFEO_TARGET=GENERIC \
-DHPIPM_TARGET=GENERIC \
-DACADOS_EXAMPLES=OFF \
-DACADOS_UNIT_TESTS=OFF
```

acados 自己支持 ACADOS_INSTALL_DIR, 所以脚本同时传递它和标准 CMAKE_INSTALL_PREFIX, 避免上游逻辑覆盖安装位置。

BLASFEO_TARGET=GENERIC 优先保证不同 CPU 可用。确认目标机具体 ARM 微架构后, 可改成上游支持的优化 target, 但不能把另一台 x86_64 机器编译出的 BLASFEO 直接复制到 ARM 无人机。

13.4 CasADi: 插件选项决定真实依赖

CasADi 可选项很多。当前关键原则是:

```
WITH_IPOPT=ON          # build CasADi's Ipopt interface
WITH_BUILD_IPOPT=OFF   # find the already installed Ipopt
WITH_SELFCONTAINED=OFF
WITH_PYTHON=OFF
```

如果启用 WITH_BUILD_IPOPT=ON, CasADi 会再下载、编译一套 Ipopt, 可能产生重复版本。self-contained 模式还可能把众多插件及其依赖复制到同一安装目录, 适合发布包, 但不适合本工程追踪每条系统依赖。

13.5 Eigen: 只有头文件, 不应寻找 libEigen.so

正确方式:

```
find_package(Eigen3 CONFIG REQUIRED)
target_link_libraries(my_target PRIVATE Eigen3::Eigen)
```

这里的“link”主要传播 include 目录和编译要求。找不到 Eigen/Eigen 通常意味着 Eigen 未安装或 include 根目录没有指向 .../include/eigen3。

13.6 NLOpt: 上游库名与发行版拆包可能不同

NLOpt 2.7.1 上游 CMake 安装 libnlopt.so 并导出 NLOpt::nlopt。某些 Debian 包另有 libnlopt_cxx.so，但不能把发行版拆包布局硬编码成上游源码安装规则。当前使用：

```
find_package(NLOpt 2.7 CONFIG REQUIRED)
target_link_libraries(my_target PRIVATE NLOpt::nlopt)
```

13.7 SuiteSparse: 使用发行版传递依赖

UMFPACK 实际依赖 AMD、CHOLMOD、SuiteSparseConfig 和 BLAS，CHOLMOD 又依赖 METIS 等组件。应安装完整开发包：

```
sudo apt install libsuitesparse-dev
```

不应从另一台机器只复制 libumfpack.so，否则非常容易缺少同版本 SONAME 和传递库。

14 自定义前缀的完整示例

14.1 统一安装到单独目录

假设使用：

```
/opt/manycontroller-deps
```

运行统一脚本：

```
PREFIX=/opt/manycontroller-deps JOBS=2 ./3rdpart/build_all.sh
```

构建 px4ctrl：

```
source /opt/ros/humble/setup.bash
colcon build --packages-up-to px4ctrl --cmake-clean-cache \
  --cmake-args \
  -DCMAKE_PREFIX_PATH=/opt/manycontroller-deps
```

如果某个包配置位置仍不标准，可以补充：

```
--cmake-args \
  -DCMAKE_PREFIX_PATH=/opt/manycontroller-deps \
  -Dacados_DIR=/opt/manycontroller-deps/cmake \
  -Dosqp_DIR=/opt/manycontroller-deps/lib/cmake/osqp
```

14.2 运行时共享库搜索

CMake 找到库只解决编译/链接阶段，Linux 动态加载器还必须在运行时找到 `.so`。

临时方法：

```
export LD_LIBRARY_PATH=/opt/manycontroller-deps/lib:$LD_LIBRARY_PATH
```

系统级方法：

```
echo /opt/manycontroller-deps/lib | \
  sudo tee /etc/ld.so.conf.d/manycontroller-deps.conf
sudo ldconfig
```

也可以为目标设置 `RPATH`，但不建议写死个人主目录。发布到固定设备路径时可使用：

```
set_target_properties(my_target PROPERTIES
  INSTALL_RPATH "/opt/manycontroller-deps/lib"
)
```

14.3 工具链文件统一管理前缀

需要多台无人机或交叉编译时，可以创建项目工具链/初始缓存文件：

```
# manycontroller-deps.cmake
list(PREPEND CMAKE_PREFIX_PATH "/opt/manycontroller-deps")
set(acados_DIR "/opt/manycontroller-deps/cmake" CACHE PATH "")
```

调用：

```
colcon build ... --cmake-args \
  -C /path/to/manycontroller-deps.cmake
```

交叉编译时还需要 `CMAKE_SYSROOT`、`CMAKE_FIND_ROOT_PATH` 和交叉编译器设置；不能把主机编译的共享库当作目标机库。

15 当前 CMake 目标与后端耦合

15.1 三个最终节点

当前数值链接关系为：

目标	直接数值依赖
<code>px4ctrl_node</code>	Eigen、OSQP、NLOpt、Ipopt、CasADi bridge、项目 <code>acados</code> solver
<code>gpttraj_visualizer_node</code>	Eigen、OSQP

px4ctrlrate_node Eigen、UMFPACK

15.2 为什么只运行 acados 仍要安装 Ipopt/NLopt/CasADi

控制器在源码中通过编译期宏选择，但 px4ctrl_node 的 source list 当前仍包含：

```
solver_nmpc_ipopt_eigen.cpp
solver_nmpc_nlopt_eigen.cpp
solver_nmpc_casadi.cpp
solver_nmpc_acados.cpp
```

链接列表也包含所有后端。因此配置和链接阶段要求全部依赖存在。更好的后续结构是：

```
option(PX4CTRL_WITH_IPOPT "Build Ipopt backend" ON)
option(PX4CTRL_WITH_NLOPT "Build NLopt backend" ON)
option(PX4CTRL_WITH_CASADI "Build CasADi backend" ON)
option(PX4CTRL_WITH_ACADOS "Build acados backend" ON)
```

然后按选项条件性执行 find_package、添加源文件和链接库。这样如果无人机只需要 acados，可避免安装 Ipopt、MUMPS、NLopt 和 CasADi。这是解除工程构建耦合，不是修改求解器内部依赖。

16 Python 计算、代码生成和仿真依赖

16.1 MuJoCo 仿真节点

quadsim_mujoco 使用：

```
quadsim_mujoco
|- NumPy
|- SciPy
|  |- scipy.integrate.solve_ivp
|  `-- scipy.spatial.transform.Rotation
|- MuJoCo physics engine
|- GLFW window/OpenGL context
|- cv_bridge -> OpenCV
`-- ROS 2 Python rclpy
```

这些依赖目前没有完整写进该 Python 包的 install_requires；setup.py 只声明了 setuptools。因此迁移仿真环境时需要单独维护 Python requirements 或 rosdep 规则。

16.2 噪声分析脚本

script/noise_analys.py 使用 NumPy、SciPy optimize/sparse/interpolate、Pandas、Matplotlib 和 scikit-learn。它们属于离线分析环境，不是飞控节点的 C++ 运行时依赖。

16.3 acados 重新生成环境

`generate_px4ctrl_acados_nmpc.py` 需要 Python CasADi、NumPy 和 `acados_template`；后者还依赖 SciPy、Matplotlib、Cython、Deprecated 等 Python 包。它们只在改变模型、代价或约束后重新生成 C 代码时使用。

已提交的 C 代码可以直接由 CMake 编译，因此正常编译 `px4ctrl` 不需要 Python CasADi 或 `acados_template`。

17 存在于源码但当前没有启用的计算库

17.1 CasADi 可选生态

CasADi 支持但当前关闭的开源后端包括：

OSQP, qpOASES, HPIPM, BLASFEO, Fatrop, SuperSCS,
Clarabel, ProxQP, DSDP, Bonmin, CBC, CLP, HiGHS,
DAQP, SLEQP, Alpaqa, HSL, SPRAL, OOQP, SQIC, SLICOT

还包含 CPLEX、Gurobi、Knitro、WORHP、SNOPT 等商业或外部接口。源码中有 `FindXXX.cmake` 或 `WITH_XXX` 选项，不代表当前机器必须安装它们。

17.2 acados 可选后端

当前关闭：

qpOASES, DAQP, HPMP, QORE, OOQP,
qpDUNES, OSQP, Clarabel, OpenMP

17.3 Eigen 测试中出现的库

Eigen 的 `tests/benchmarks` 会探测 CHOLMOD、UMFPACK、KLU、SuperLU、PaStiX、SPQR、FFTW、CUDA、HIP、Boost 等。当前以 `BUILD_TESTING=OFF` 安装 Eigen 头文件，所以这些探测项不是 Eigen 的实际运行依赖。

一般判断规则为：

源码中出现库名 $\nRightarrow$ 当前构建启用该库

(2)

应同时检查 CMake cache、构建选项、最终链接命令和 `ELF_NEEDED` 项。

18 常见错误的根因与处理

18.1 导入目标引用不存在的绝对路径

错误示例：


```
The imported target "casadi::casadi" references
/home/.../3rdpart/casadi/libcasadi.so.3.7
but this file does not exist.
```

原因通常是复制了旧安装目录或 CMake cache，其中包配置记录了另一台机器的绝对路径。处理：

1. 在目标机重新源码编译并安装；
2. 删除旧 CMake cache；
3. 使用 `--cmake-clean-cache`；
4. 检查 `casadi_DIR` 和 `CMAKE_PREFIX_PATH`。

18.2 QDLDL 更新步骤 rebase 失败

```
fatal: invalid upstream 'origin/v0.1.8'
Failed to rebase in qdldl-src
```

标签 `v0.1.8` 被更新脚本误当成远端分支。当前通过固定 `QDLDL` 子模块、设置 `FETCHCONTENT_SOURCE_DIR_QDLDL` 和禁止更新解决。

18.3 Eigen/Eigen 找不到

```
fatal error: Eigen/Eigen: No such file or directory
```

Eigen 的典型布局是 `PREFIX/include/eigen3/Eigen`。使用 `Eigen3::Eigen` 可以自动传播正确 `include` 根目录；不要只把 `PREFIX/include` 当成 Eigen `include` 目录。

18.4 qpOASES.hpp 找不到

CasADi 的 qpOASES 插件或插件共享库不等于系统安装了 qpOASES C++ 开发头文件。当前 `px4ctrl` 不调用 qpOASES API，历史遗留 `include` 已删除，所以不应为此额外安装 qpOASES。

18.5 GLIBC 符号版本不匹配

```
undefined reference to pthread_join@GLIBC_2.34
undefined reference to fstat@GLIBC_2.33
```

这说明复制来的共享库在更新的 glibc 上编译。添加 `-pthread` 或 `include` 路径不能修复 ABI 版本不匹配；必须在目标机原生编译，或使用兼容 `sysroot`/容器构建。

18.6 编译能找到，运行时找不到

```
error while loading shared libraries: libacados.so: cannot open...
```

检查：

```
ldconfig -p | grep acados
ldd /path/to/px4ctrl_node
readelf -d /path/to/px4ctrl_node | grep NEEDED
```

标准 `/usr/local/lib` 安装后执行 `sudo ldconfig`；自定义目录则配置 `ld.so`、`LD_LIBRARY_PATH` 或合适 `RPATH`。

19 新机器从零构建流程

19.1 安装系统依赖

```
sudo apt update
sudo apt install -y \
  build-essential cmake pkg-config autoconf automake libtool \
  gfortran libblas-dev liblapack-dev libmumps-seq-dev \
  libscotch-dev libmetis-dev libsuitesparse-dev \
  python3-colcon-common-extensions
```

19.2 同步固定源码

```
git pull --ff-only
git submodule sync --recursive
git submodule update --init
git -C 3rdpart/src/acados submodule update --init \
  external/blasfeo external/hpipm
```

不建议直接使用顶层 `--recursive` 初始化 `acados` 的所有可选子模块，因为会下载当前没有启用的 `qpOASES`、`OSQP`、`DAQP`、`Clarabel` 等项目。

19.3 安装到 `/usr/local`

无人机内存有限时降低并行数：

```
JOBS=2 ./3rdpart/build_all.sh
```

19.4 编译工作空间

```
source /opt/ros/humble/setup.bash
colcon build --packages-up-to px4ctrl --cmake-clean-cache
```

20 验证依赖是否正确

20.1 确认子模块提交

```
git submodule status
git -C 3rdpart/src/acados submodule status \
    external/blasfeo external/hpipm
```

状态行前应为空格；- 表示未初始化，+ 表示工作区提交与主仓库记录不一致。

20.2 确认 CMake 包配置

```
find /usr/local -type f \
    \( -name '*Config.cmake' -o -name '*-config.cmake' \) \
    | grep -E 'Eigen|osqp|NLOPT|casadi|acados|hpipm|blasfeo'
```

20.3 确认头文件和共享库

```
find /usr/local/include -maxdepth 4 \
    \( -name Eigen -o -name osqp.h \
        -o -name IpIpoptApplication.hpp \
        -o -name ocp_nlp_interface.h \)

ldconfig -p | grep -E \
    'casadi|acados|hpipm|blasfeo|osqp|ipopt|nlopt|umfpack'
```

20.4 确认最终程序没有引用仓库旧二进制

```
ldd install/px4ctrl/lib/px4ctrl/px4ctrl_node
readelf -d install/px4ctrl/lib/px4ctrl/px4ctrl_node \
    | grep -E 'NEEDED|RPATH|RUNPATH'
```

输出不应再出现 /home/.../3rdpart/casadi、3rdpart/optimization 或其他机器的绝对路径。

21 推荐的长期工程结构

21.1 保持源码、构建结果和安装结果分离

```
repository/
|- 3rdpart/src/          fixed upstream source checkouts
|- 3rdpart/build/        local build trees, ignored by Git
|- src/.../generated/    project-owned generated source
`- /usr/local            installed libraries outside repository
```

自定义前缀时：

```
repository/          source only
/opt/manycontroller-deps/  installed headers/libs/configs
```

不要重新把安装后的 `include/`、`lib/*.so`、`share/doc` 作为普通文件提交到 3rdpart。

21.2 每个依赖明确记录四个信息

建议对每个第三方项目记录：

1. 来源 URL 和固定 commit/tag；
2. 构建系统及关键选项；
3. 安装前缀和提供的 CMake target；
4. 直接依赖、传递依赖和运行时加载方式。

21.3 将后端按需编译

长期建议拆分 px4ctrl 的求解器后端。只启用 acados 时，原生依赖可以缩小为：

```
Eigen
OSQP          # still needed by gptmpc/gpttraj if those sources remain
acados
HPIPM
BLASFEO
SuiteSparse/UMFPACK
```

如果同时禁用 GptMpc/GptTraj 和 rate controller 中相关功能，还能进一步减少 OSQP 或 SuiteSparse；但必须按实际目标 source list 和运行功能决定，不能仅凭控制器运行时宏删除依赖。

22 变量速查表

变量/命令	作用	示例
CMAKE_INSTALL_PREFIX	当前库安装到哪里	/usr/local、/opt/deps
CMAKE_PREFIX_PATH	主工程去哪些前缀找包	/opt/deps-a;/opt/deps-b
Foo_DIR	精确指定 FooConfig.cmake 目录	osqp_DIR=/opt/deps/lib/cmake/osqp
Foo_ROOT	项目自定义的某库前缀提示	IPOPT_ROOT=/opt/ipopt
CMAKE_INCLUDE_PATH	为 find_path 增加全局头文件目录	/opt/legacy/include

变量/命令	作用	示例
CMAKE_LIBRARY_PATH	为 find_library 增加全局库目录	/opt/legacy/lib
FETCHCONTENT_SOURCE_DIR_QDLDL	OSQP 使用本地 QDLDL 源码	3rdpart/src/qdldl
ACADOS_INSTALL_DIR	acados 上游专用安装变量	通常等于安装前缀
BLASFEO_TARGET	BLASFEO CPU kernel 目标	GENERIC 或上游架构值
LD_LIBRARY_PATH	临时运行时共享库搜索路径	/opt/deps/lib
INSTALL_RPATH	把运行时库路径写入目标	固定部署目录时使用
PKG_CONFIG_PATH	pkg-config 查找额外前缀	/opt/deps/lib/pkgconfig
--cmake-clean-cache	丢弃旧绝对路径和查找结果	迁移机器后必须使用

23 最终原则

本工程的第三方依赖管理可以归纳为以下规则：

1. 标准安装包使用 find_package 和导入 target；
2. 没有包配置的遗留库使用 find_path/find_library，最好再封装为导入 target；
3. 项目自己的生成源码使用 add_library，不提交生成二进制；
4. 构建期工具使用 find_program/add_custom_command，不能当库链接；
5. 大型或特殊构建系统按上游风格独立编译安装；
6. Git 子模块只负责固定源码版本，不等于已经安装；
7. CMAKE_INSTALL_PREFIX 管安装位置，CMAKE_PREFIX_PATH 管查找位置；
8. /usr/local 只是系统默认认识的标准前缀，自定义目录同样可行；
9. 源码中出现可选库不代表当前启用，必须看构建选项和最终 ELF 依赖；
10. 不同 CPU、glibc 和 C++ ABI 的机器应从源码重新构建，不能依赖复制来的共享库。